

BAGIAN 1 DARI 2 • PERTEMUAN 3

Dasar-dasar Machine Learning

Terminologi dan Teknik Dasar

Cakupan materi ini

- ① Permasalahan Pembelajaran — target, tujuan, dan data
- ② Perancangan Model — keluarga model, ruang hipotesis, dan bias induktif

Tujuan Pembelajaran

Setelah materi ini, mahasiswa diharapkan mampu:

1

Merumuskan

Menerjemahkan persoalan nyata menjadi permasalahan pembelajaran: apa targetnya, apa ukuran keberhasilannya, dan data apa yang tersedia.

2

Membedakan

Mengenali jenis tugas ML — klasifikasi, regresi, klustering, reduksi dimensi — beserta ciri datanya.

3

Menjelaskan

Menerangkan generalisasi dan alasan pemisahan data latih, validasi, dan uji.

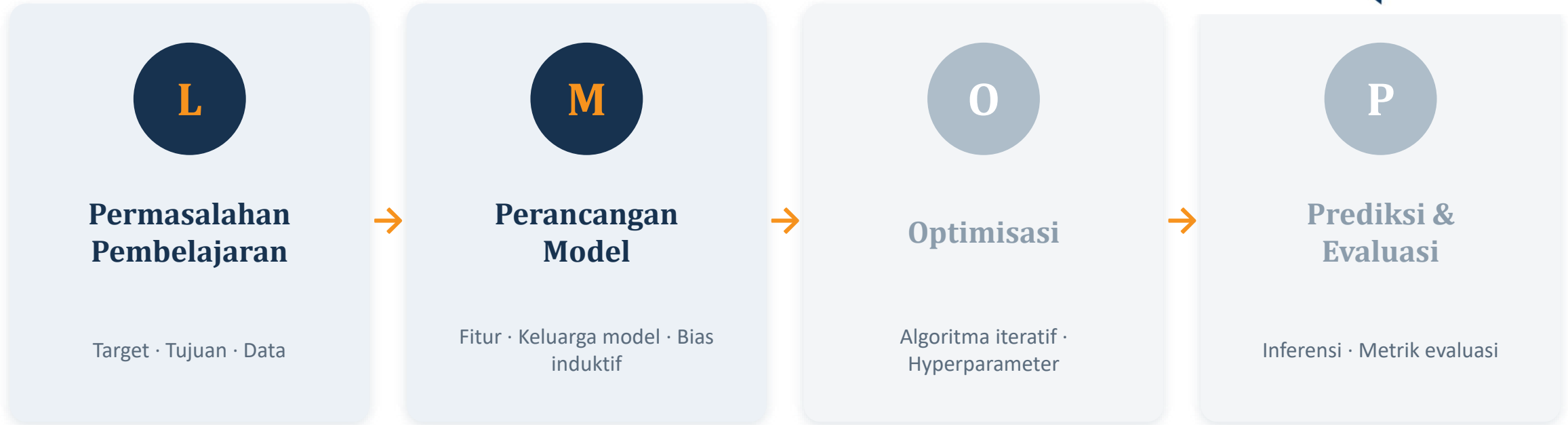
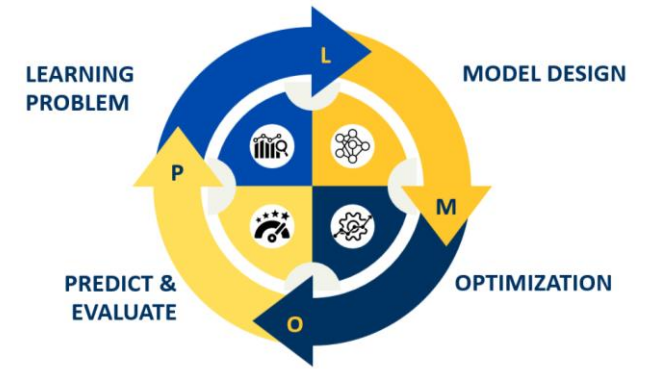
4

Merancang

Memilih fitur, keluarga model, dan tingkat kompleksitas dengan mempertimbangkan bias induktif.

Siklus Hidup Machine Learning

Empat tahap yang selalu berulang. Materi ini membahas dua tahap pertama.



Tahap Optimisasi dan Prediksi & Evaluasi dibahas pada materi berikutnya: “Terminologi dan Teknik Dasar (2)”.

Perkakas Kita: Scikit-Learn

Satu pustaka Python yang menemani seluruh siklus hidup ML klasik.

Persiapan data

Pembagian data, penanganan nilai hilang, penskalaan.

Model klasik

Regresi linear/logistik, pohon keputusan, k-NN, SVM.

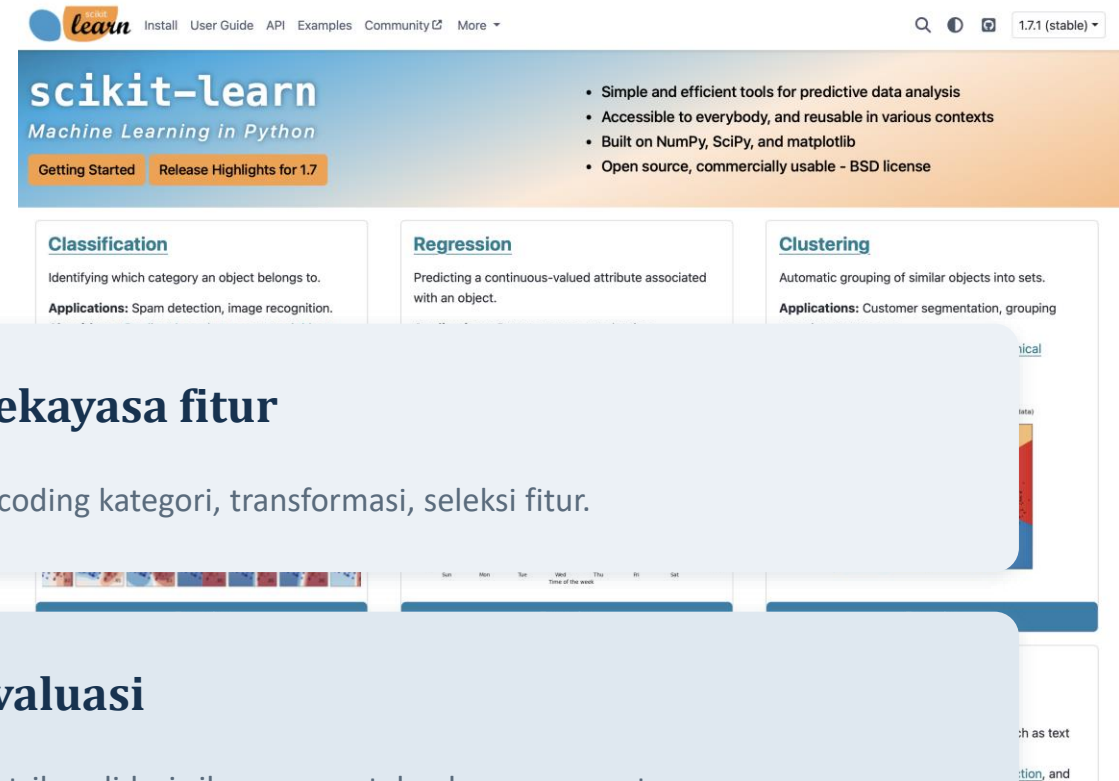
Rekayasa fitur

Encoding kategori, transformasi, seleksi fitur.

Evaluasi

Metrik, validasi silang, penyetelan hyperparameter.

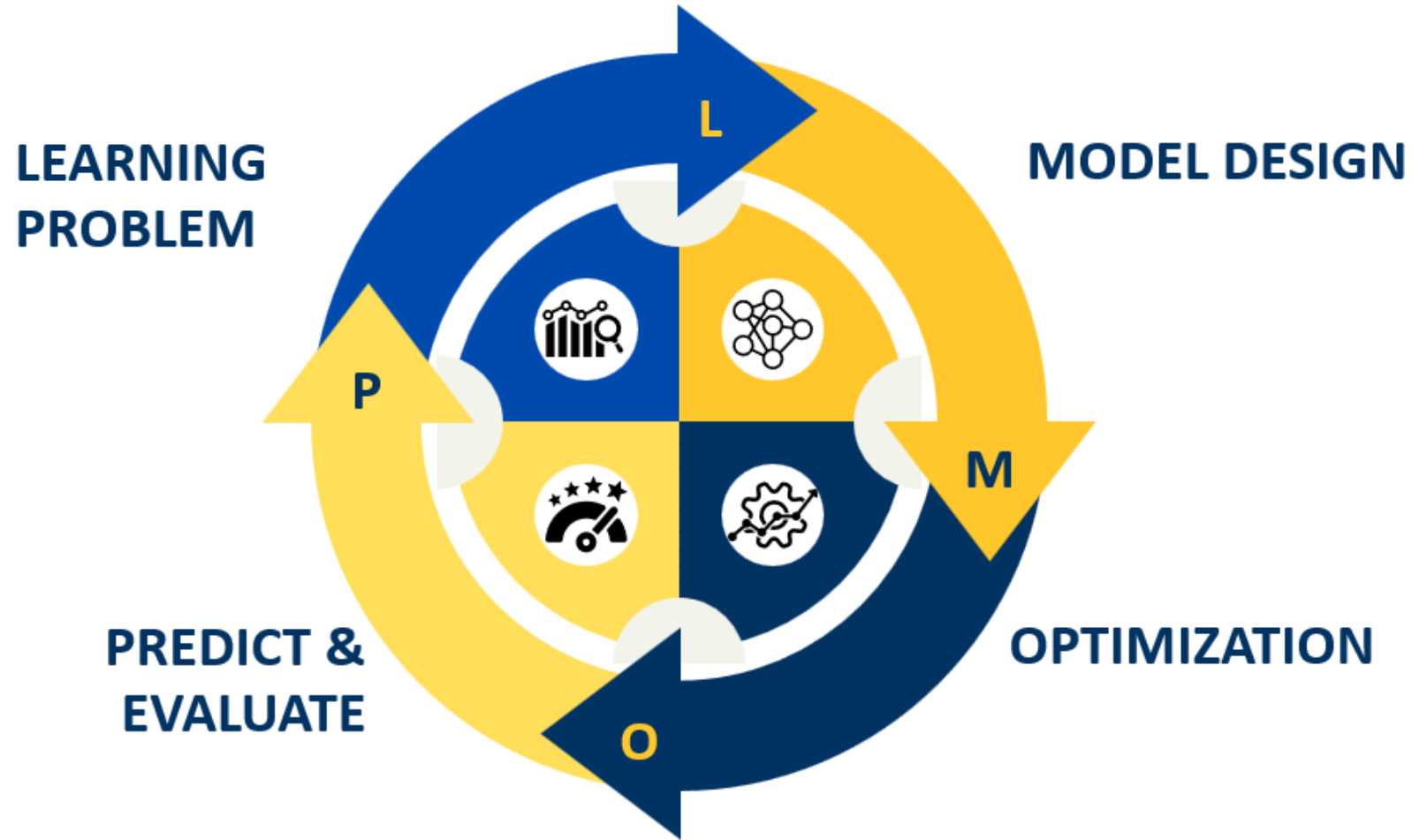
```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
```



ML Lifecycle



282609



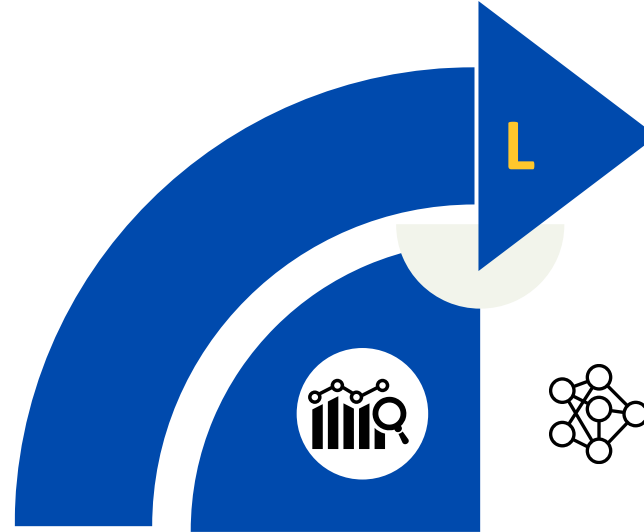
ML Lifecycle



2826097

LEARNING PROBLEM

- Target
- Objective
- Data



BAGIAN 01

Permasalahan Pembelajaran

Sebelum menyentuh model, rumuskan dulu masalahnya dengan benar.

● 1. Permasalahan Pembelajaran

● 2. Perancangan Model

LEARNING PROBLEM



Tiga Pertanyaan Pembuka

Setiap proyek ML dimulai dari tiga pertanyaan ini, dan bukan dari memilih algoritma.

1

Target

- Apa yang ingin saya prediksi?
- Jenis tugas apa ini — klasifikasi, regresi, klustering, atau reduksi dimensi?

2

Tujuan (Objective)

- Bagaimana saya menilai keberhasilan model?
- Fungsi kerugian (loss) apa yang mengukur kesalahan model saat dilatih?

3

Data

- Data apa yang saya miliki?
- Bagaimana data direpresentasikan menjadi fitur?
- Bagaimana pembagian data latih dan uji?

Kesalahan yang paling mahal dalam proyek ML bukan salah memilih algoritma, melainkan salah merumuskan masalah.

Studi Kasus: FashionHub

Contoh yang akan kita pakai berulang kali sepanjang dua materi ini.

Situasinya

Sebuah situs jual-beli fashion baru diluncurkan. Pengguna mengunggah foto pakaian yang ingin ditukar, dan sistem harus memberi kategori pakaian itu secara otomatis. Kita sudah punya sejumlah foto pakaian beserta label kategorinya.

Target

Jenis pakaian pada foto

Klasifikasi multi-kelas

Tujuan

Akurasi keseluruhan

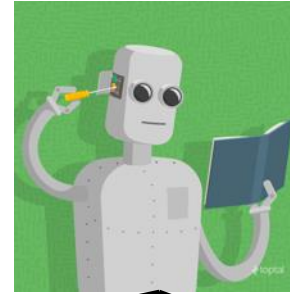
Cross-entropy sebagai loss

Data

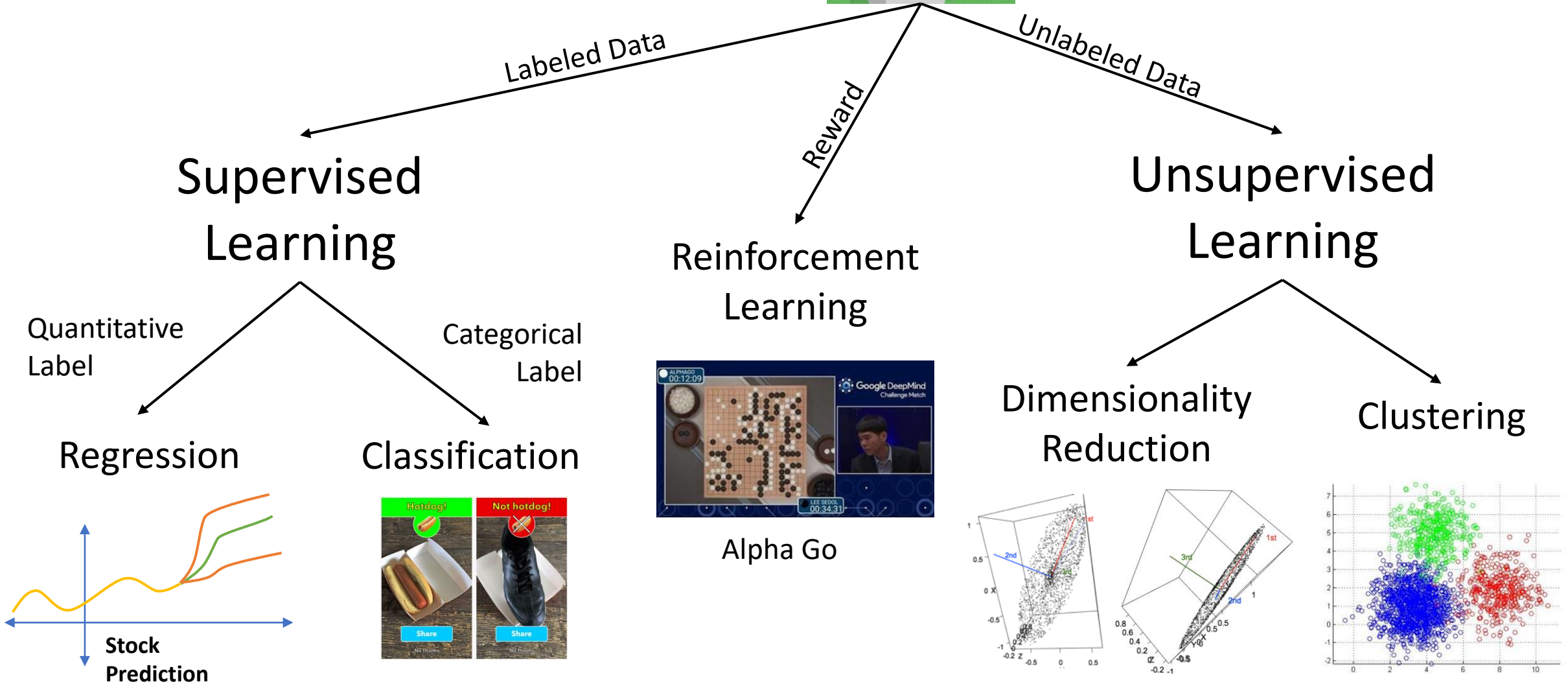
Foto pakaian berlabel

Piksel gambar sebagai fitur

Taxonomy of Machine Learning



2826097



Taksonomi Machine Learning

Jenis data yang tersedia menentukan jenis pembelajaran yang mungkin dilakukan.

Data berlabel

Supervised Learning

- Klasifikasi — label kategorikal (jenis pakaian)
- Regresi — label kuantitatif (harga saham)

Data tanpa label

Unsupervised Learning

- Klastering — mengelompokkan data serupa
- Reduksi dimensi — memadatkan fitur

Berbasis imbalan

Reinforcement Learning

- Agen belajar dari percobaan dan imbalan
- Contoh: AlphaGo, robotika, permainan

Self-supervised learning (SSL): teknik mempelajari representasi berguna dari data tanpa label, misalnya dengan menutup sebagian data lalu memintanya diprediksi. Menjadi dasar model bahasa dan visi modern.

Supervised Learning: Belajar dari Contoh

Cara belajar yang paling banyak dipakai di industri.

Kelebihan

- Belajar cepat karena hubungan input–output ditunjukkan langsung.
- Butuh data relatif lebih sedikit dibandingkan pendekatan lain.

Kelemahan

- Harus tersedia contoh berlabel yang cukup banyak.
- Pelabelan sering mahal, lambat, dan rawan kesalahan manusia.

Klasifikasi

Label berupa kategori diskret.

Contoh: jenis pakaian pada foto, email spam atau bukan.

Regresi

Label berupa angka kontinu.

Contoh: harga rumah, suhu besok, harga saham.

Masalah Klasifikasi Lebih Dekat

Label diskret sering diberi kode angka, tetapi angkanya tidak punya urutan.

Pengodean label

0 = Kaos 1 = Celana panjang 2 = Sweater

Angka 2 bukan berarti “dua kali lebih besar” daripada 1. Angka hanya berfungsi sebagai nama kelas.

Klasifikasi biner

Tepat dua kelas. Spam atau bukan spam.

Klasifikasi multi-kelas

Lebih dari dua kelas. Hotdog, Pizza, Burrito, ...

Mengapa ini penting?

- Jika label diberi kode 0/1/2 lalu diperlakukan sebagai angka biasa, model akan mengira Sweater “lebih besar” daripada Kaos.
- Karena itu label kategorikal ditangani sebagai kelas, bukan bilangan — dan fitur kategorikal diubah dengan one-hot encoding.
- Jenis tugas juga menentukan fungsi kerugian: cross-entropy untuk klasifikasi, kuadrat galat untuk regresi.
- Jumlah kelas memengaruhi pilihan metrik: akurasi saja sering menyesatkan pada kelas yang tidak seimbang.

Tujuan (Objective): Ukuran Keberhasilan

Dua hal berbeda yang sering tertukar.

Fungsi kerugian (loss)

Dipakai SAAT PELATIHAN.

- Mengukur kesalahan model pada setiap contoh.
- Harus dapat dioptimalkan secara matematis (biasanya diferensiabel).
- Contoh: cross-entropy, mean squared error.

Metrik evaluasi

Dipakai SAAT PENILAIAN.

- Mengukur keberhasilan menurut kebutuhan bisnis.
- Boleh tidak diferensiabel dan mudah dibaca manusia.
- Contoh: akurasi, precision, recall, RMSE.

Pada FashionHub: model dilatih meminimalkan cross-entropy, tetapi keberhasilannya kita laporkan sebagai akurasi kategori.

Data: Lihat Datanya Dulu!

Sebelum melatih apa pun, jawab daftar periksa berikut.

Tentang data secara umum

- Berapa banyak data yang ada? ($N = ?$)
- Berapa banyak fitur atau dimensi? ($D = ?$)
- Apakah semuanya numerik?
- Adakah nilai yang hilang?

Tentang fitur

- Apa arti setiap fitur?
- Bagaimana distribusinya — normal, miring, banyak pencilan?
- Apakah skalanya sangat berbeda antarfitur?

Tentang label

- Apakah ada label? Diskret atau kontinu?
- Bagaimana distribusi label — seimbang atau timpang?
- Adakah label yang hilang atau salah?

Perhatikan data dengan saksama: ambil sampel, baca isinya, buat grafiknya. Jangan langsung melatih model.

Mengapa Menghafal Data Itu Buruk?

Analogi ujian: apa jadinya bila semua orang diberi soal ujian beserta kuncinya sebelum ujian?

Apakah semua orang akan mendapat nilai tinggi?

Ya — hampir pasti.

Apakah itu berarti mereka menguasai materi?

Belum tentu sama sekali.

Apa yang terjadi bila soalnya diganti?

Nilai mereka akan anjlok.

Overfitting

Model yang hanya mengingat contoh pelatihan akan tampak sangat baik di data latih, tetapi gagal pada data baru. Ia menghafal derau, bukan mempelajari pola.

Tujuan machine learning bukanlah mencocokkan data latih, melainkan membuat prediksi akurat pada data yang belum pernah dilihat.

Generalisasi

Kemampuan model bekerja baik pada data baru yang berasal dari pola yang sama.

Generalisasi adalah kemampuan sebuah model untuk memberikan prediksi yang baik pada data yang belum pernah dilihat, sepanjang data itu berasal dari sumber atau distribusi yang sama dengan data pelatihan.

Cocok di data latih

Mudah dicapai. Model cukup dibuat serumit mungkin.

Bukan tujuan

Cocok di data baru

Inilah yang benar-benar kita inginkan dari sebuah model.

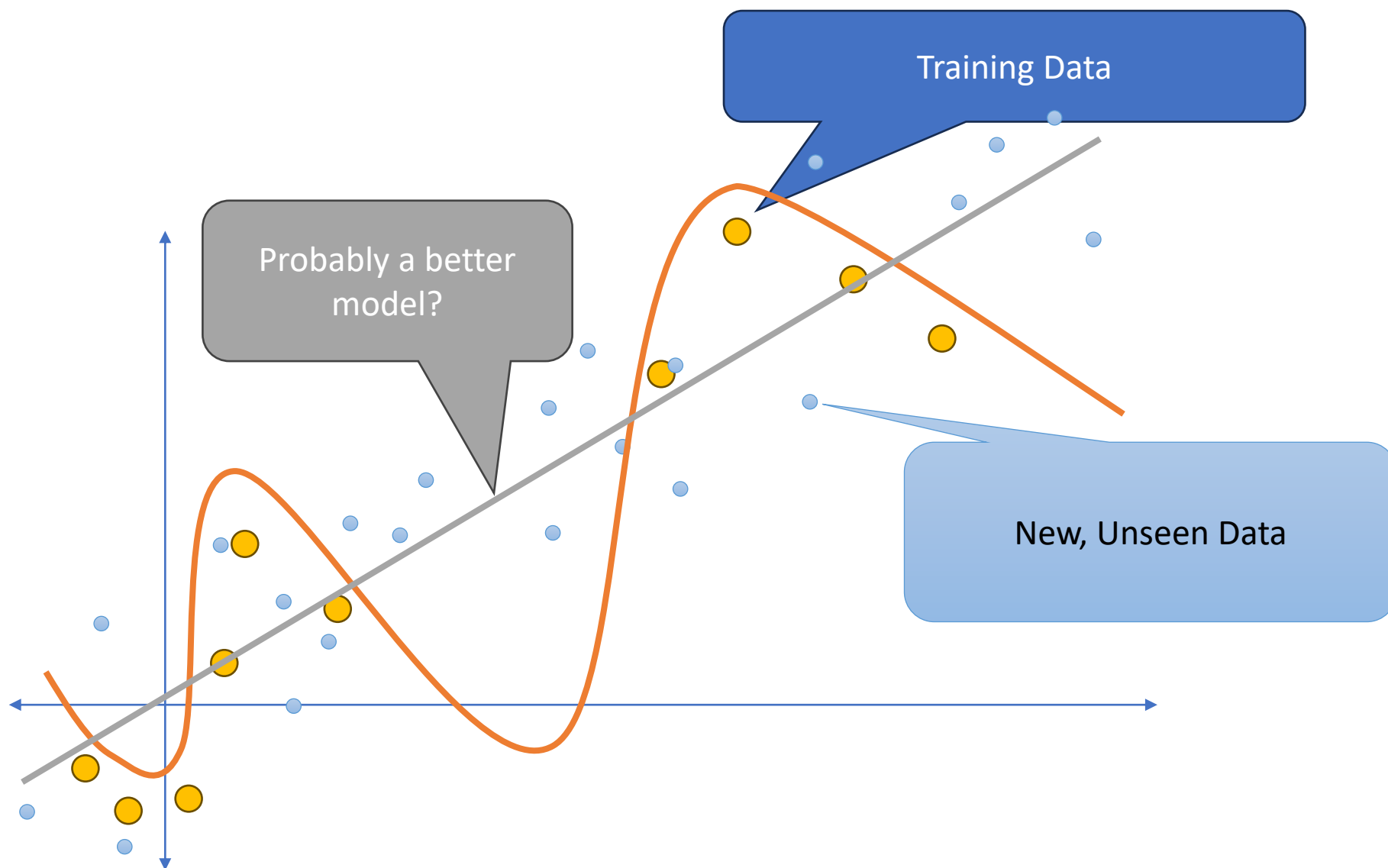
Tujuan sebenarnya

Cara mengukurnya

Sisihkan sebagian data dan jangan pernah dipakai melatih.

Train–test split

Perlu diingat: jika data baru berasal dari distribusi yang berbeda, generalisasi tidak lagi dijamin.



Train-Test Split

Teknik baku untuk mengukur generalisasi.

1

Acak data

Kocok data terlebih dahulu agar pembagian tidak dipengaruhi urutan penyimpanan.

2

Bagi dua

Bagian latih $\pm 80\%$ untuk mengembangkan dan melatih model.

3

Sisihkan uji

Bagian uji $\pm 20\%$ khusus untuk menilai kemampuan generalisasi.

```
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2,  
random_state=42)
```

Aturan emas

Data uji hanya boleh dipakai SEKALI, setelah model selesai dikembangkan dan dilatih.

Mengapa? Karena begitu data uji dipakai untuk menyetel model, model sudah “melihat” data itu. Nilainya berhenti mengukur generalisasi dan berubah menjadi angka yang terlalu optimistis.

Menambahkan Data Validasi

Lalu bagaimana bila kita tetap perlu menyetel model?

Train ~60%

Melatih model: menemukan nilai parameter terbaik.

Validation ~20%

Menyetel rancangan model dan hyperparameter. Boleh dipakai berkali-kali.

Test ~20%

Penilaian akhir sekali saja. Disimpan rapat sampai akhir.

Analogi belajar untuk ujian

Train

Bank soal dan pembahasan yang Anda pelajari.

Validation

Latihan ujian (try out) untuk mengecek apakah cara belajar Anda sudah efektif.

Test

Ujian sesungguhnya yang menentukan kelulusan.

Ringkasan Bagian 1

Sebelum melangkah ke perancangan model, pastikan hal-hal berikut sudah jelas.

1

Target sudah dirumuskan: apa yang diprediksi dan termasuk jenis tugas apa.

2

Jenis pembelajaran sudah ditentukan: supervised, unsupervised, atau reinforcement.

3

Ukuran keberhasilan sudah disepakati, dan dibedakan dari fungsi kerugian.

4

Data sudah benar-benar dilihat: jumlah, fitur, distribusi, nilai hilang, dan label.

5

Data sudah dibagi menjadi latih, validasi, dan uji sebelum eksperimen dimulai.

6

Data uji disimpan dan tidak disentuh sampai penilaian akhir.

BAGIAN 02

Perancangan Model

Memilih bentuk model, ruang pencarian, dan asumsi yang kita tanamkan.

- 1. Permasalahan Pembelajaran
- **2. Perancangan Model**

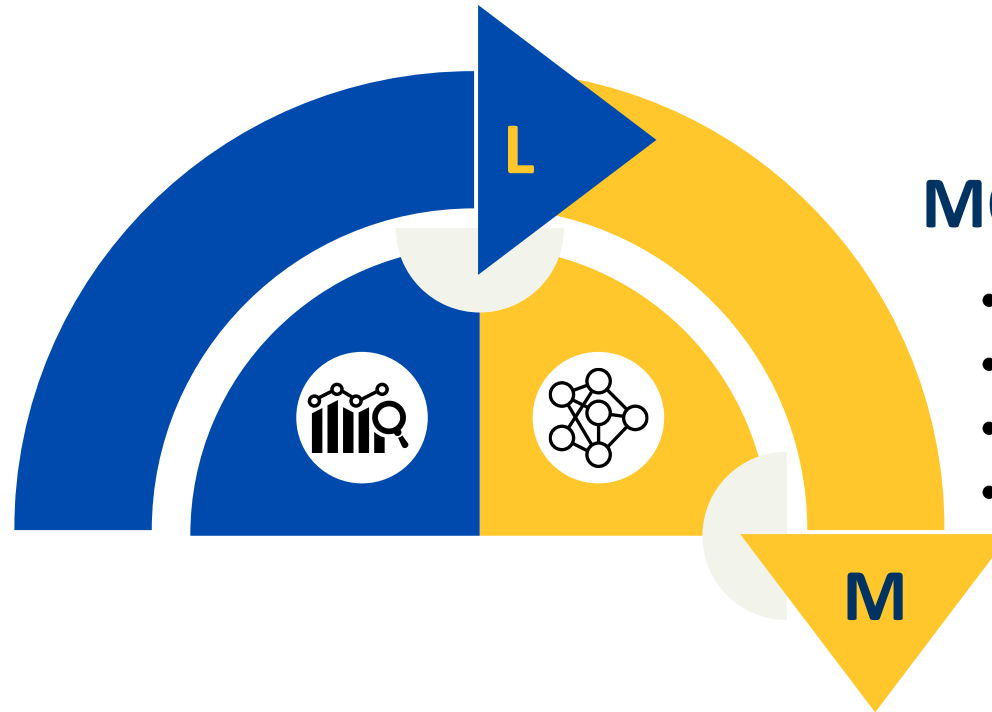


2826097

ML Lifecycle

LEARNING PROBLEM

- Target
- Objective
- Data



MODEL DESIGN

- Feature Engineering
- Model family/Architecture
- Hypothesis space
- Inductive biases / Assumptions

Empat Keputusan dalam Perancangan Model

Semuanya diambil sebelum satu baris pelatihan dijalankan.

1

Rekayasa Fitur

Memilih dan mengubah informasi dari data mentah agar dapat diproses model.

2

Keluarga Model / Arsitektur

Bentuk umum model yang dipakai: linear, pohon, jaringan saraf, dan lain-lain.

3

Ruang Hipotesis

Himpunan semua model yang mungkin dihasilkan oleh keluarga model tersebut.

4

Bias Induktif / Asumsi

Dugaan awal tentang pola data yang membuat model dapat menggeneralisasi.

Keempatnya saling terkait: fitur yang baik sering membuat keluarga model sederhana sudah cukup.

Rekayasa Fitur (Feature Engineering)

Proses memilih dan mengubah informasi dari data mentah agar dapat dipakai model.

Memilih fitur

- Fitur yang tepat sering berdampak lebih besar daripada mengganti algoritma.
- Menambah fitur dari sumber data lain dapat sangat membantu.
- Namun terlalu banyak fitur dengan data sedikit justru merugikan — model mudah overfitting.
- Ada metode otomatis untuk menyeleksi fitur penting; akan dibahas kemudian.

Mengubah fitur (encoding)

- Model hanya memahami angka, sehingga data kategori, teks, dan gambar harus diubah dahulu.
- Bentuk akhirnya selalu sama: vektor angka berdimensi tinggi.
- Inti kemajuan deep learning adalah kemampuannya mempelajari transformasi ini secara otomatis.
- Namun untuk data tabular, rekayasa fitur manual masih sangat menentukan.

Menangani Data Numerik dan Kategorikal

Angka belum tentu boleh dipakai apa adanya.

Fitur kategorikal berkode angka

Kode pos, kode produk, atau kode warna (merah = 1, biru = 2).

→ one-hot encoding

Fitur sangat miring (skewed)

Jumlah klik, harga, jumlah tayangan — rentangnya bisa sangat lebar.

→ transformasi logaritma

Fitur berbeda skala

Satu fitur bernilai 1–10, fitur lain 1.000–10.000.

→ standarisasi

Standarisasi mengubah fitur menjadi berrata-rata 0 dan berdeviasi standar 1, sehingga semua fitur berada pada skala yang sebanding.

One-Hot Encoding

Satu kolom kategori diubah menjadi beberapa kolom biner, satu kolom untuk tiap nilai.

Sebelum

Warna
Merah
Hijau
Merah
Biru
Kuning



Sesudah

Merah	Biru	Hijau	Kuning
1	0	0	0
0	0	1	0
1	0	0	0
0	1	0	0
0	0	0	1

Mengapa begini?

- Model tidak lagi menganggap kategori punya urutan atau nilai lebih besar.
- Nilai yang hilang dapat diberi kolom tersendiri, misalnya Warna:Hilang.
- Kelemahannya: jumlah kolom membengkak bila kategorinya sangat banyak.

```
from sklearn.preprocessing import OneHotEncoder
ohe = OneHotEncoder(handle_unknown='ignore')
X_enc = ohe.fit_transform(df[['warna']])
```

Mengubah Data Teks Menjadi Angka

Tiga pendekatan, dari yang paling sederhana sampai yang paling modern.

1

One-hot encoding

Untuk kategori teks bernilai terbatas: nama warna, negara, atau kota. Setiap nilai menjadi satu kolom biner.

2

Bag-of-Words

Untuk kalimat atau dokumen. Setiap kata menjadi kolom, isinya jumlah kemunculan kata itu. Urutan kata diabaikan.

3

Learned Vector Embeddings

Model bahasa besar mengubah teks menjadi vektor berdimensi tetap yang mewakili maknanya, termasuk konteks dan urutan.

Apa pun tekniknya, hasil akhirnya sama: teks berubah menjadi vektor angka berdimensi tinggi.

Bag-of-Words

Menghitung kemunculan kata tanpa memedulikan urutannya.

“Belajar tentang machine learning itu menyenangkan.”

Kata (kosakata)	Jumlah
belajar	1
learning	1
machine	1
menyenangkan	1
tentang	1
zebra	0

Hal penting

- Panjang vektor sama dengan ukuran kosakata, sehingga vektornya sangat panjang dan hampir seluruhnya nol.
- Kata umum yang tidak informatif (yang, di, dan, adalah) biasanya dibuang sebagai stop words.
- Urutan kata hilang: “anjing menggigit orang” dan “orang menggigit anjing” menghasilkan vektor yang sama.
- Keterbatasan inilah yang kemudian diatasi oleh embedding dan model berbasis Transformer.

Mengubah Data Gambar Menjadi Angka

Gambar berwarna adalah tensor tiga dimensi: tinggi $\times$ lebar $\times$ kanal warna.

Flatten

Meratakan tensor gambar menjadi satu deret angka. Paling sederhana, cukup efektif untuk gambar kecil dan monokrom.

Transformasi

Mengubah ruang warna dan menormalkan nilai piksel agar lebih konsisten antar-gambar.

Fitur buatan manual

Menggunakan pendeteksi tepi atau deskriptor tekstur untuk mengambil informasi penting.

Representasi deep learning

Jaringan saraf mempelajari sendiri embedding gambar yang paling berguna.

Skalar (0 dimensi) $\rightarrow$ vektor (1) $\rightarrow$ matriks (2) $\rightarrow$ tensor (3 atau lebih). Gambar RGB berukuran 28 $\times$ 28 adalah tensor 28 $\times$ 28 $\times$ 3.

Pola Baku Scikit-Learn: fit dan transform

Hampir semua alat pengubah data di Scikit-Learn memakai pola yang sama.

```
from sklearn.preprocessing import OneHotEncoder

ohe = OneHotEncoder()           # 1. Konstruktor
ohe.fit(df[['warna']])          # 2. Pelajari kategorinya
ohe.transform(df[['warna']])    # 3. Ubah datanya
```

fit

Mempelajari parameter transformasi dari data — daftar kategori, rata-rata, deviasi standar.

transform

Menerapkan transformasi itu pada data.

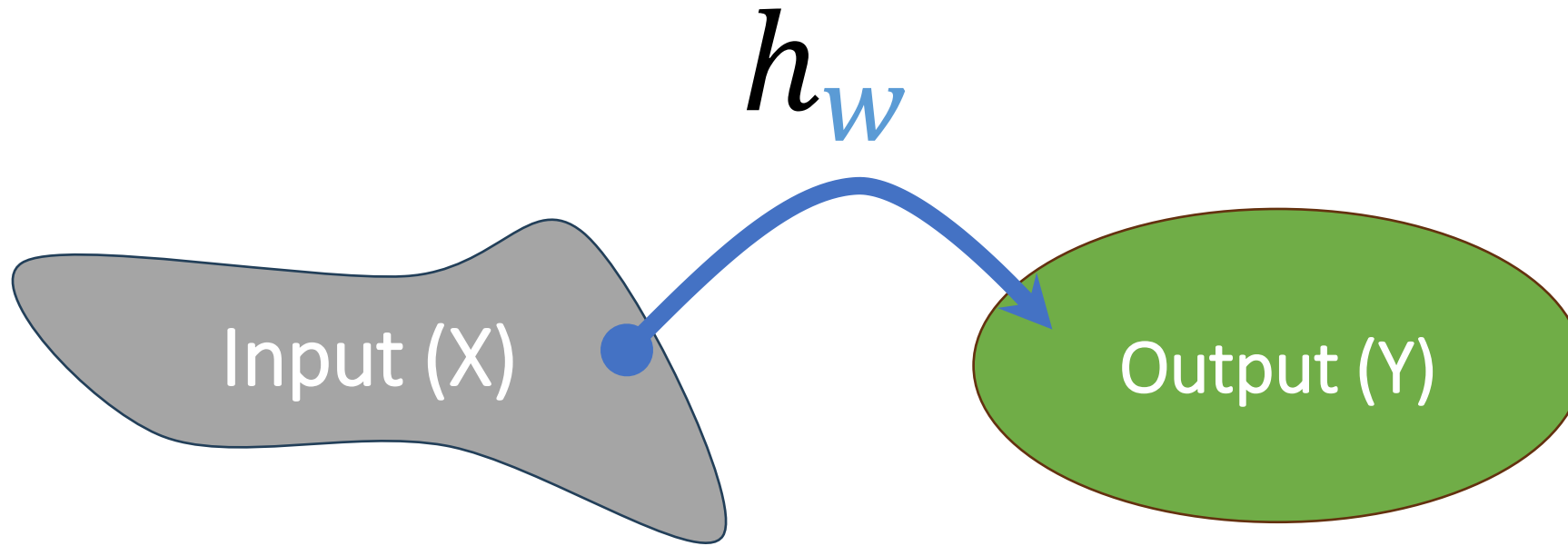
fit_transform

Menggabungkan keduanya. Hanya untuk data latih.

Kesalahan yang sangat sering terjadi

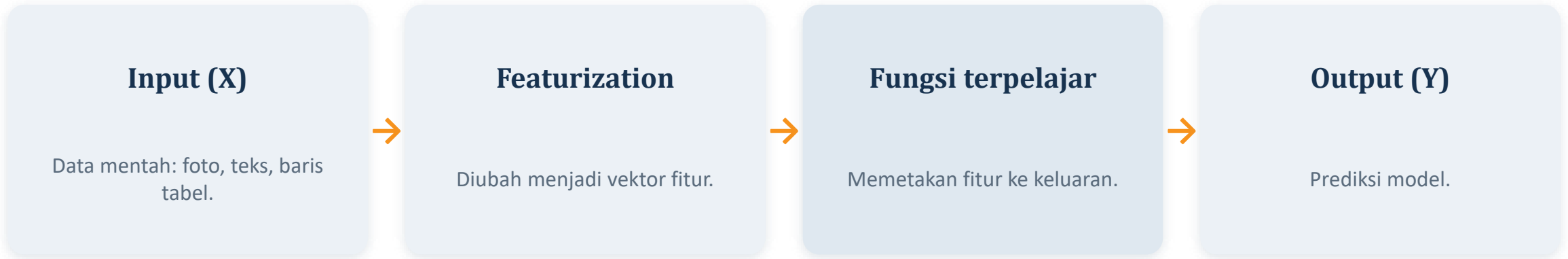
Melakukan fit pada seluruh data sebelum pembagian latih–uji. Akibatnya informasi dari data uji ikut bocor ke dalam transformasi (data leakage), dan hasil evaluasi menjadi terlalu bagus. Aturannya: fit hanya pada data latih, lalu transform pada data validasi dan uji.

Machine Learning
as **Function Approximation**



Machine Learning sebagai Aproksimasi Fungsi

Cara pandang yang menyatukan hampir semua metode ML.

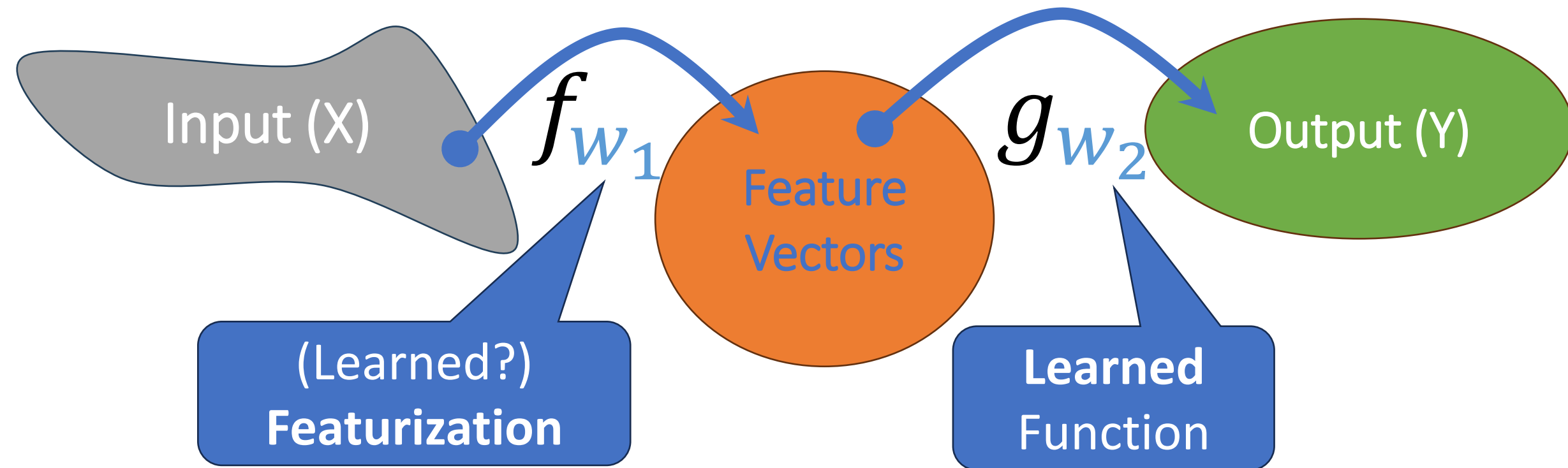


Apa maksudnya “aproksimasi”?

- Kita berasumsi ada fungsi tersembunyi yang menghubungkan input dengan output, lalu berusaha mendekatinya dari data.
- Fungsi hasil pembelajaran tidak akan sempurna — cukup mendekati agar prediksinya berguna.
- Pada deep learning, tahap featurization pun ikut dipelajari, bukan dirancang manual.

Machine Learning
as **Function Approximation** with
(Learned) **Features**

$$h_w = g_{w_2} \circ f_{w_1}$$



Keluarga Model dan Ruang Hipotesis

Dua istilah yang sering tertukar, padahal hubungannya sederhana.

1

Keluarga Model / Arsitektur

Bentuk umum fungsi yang kita pilih.

Contoh: “semua garis lurus $y = w \cdot x + b$ ”, pohon keputusan, atau jaringan saraf dengan arsitektur tertentu.

2

Ruang Hipotesis

Himpunan semua model konkret yang dapat dihasilkan keluarga itu.

Untuk regresi linear: kumpulan seluruh garis lurus yang mungkin, dengan segala nilai kemiringan dan titik potong.

Melatih model = mencari satu titik terbaik di dalam ruang hipotesis. Memilih keluarga model = menentukan seberapa luas ruang itu.

Bias Induktif (Inductive Bias)

Asumsi yang sengaja kita tanamkan agar model dapat menggeneralisasi.

Dengan data yang terbatas, ada tak berhingga banyak model yang sama-sama cocok dengan data latih — dan semuanya “benar” menurut data itu. Bias induktif adalah petunjuk awal yang membuat model memilih pola yang masuk akal ketika bertemu data baru.

Regresi linear

Mengasumsikan hubungan input dan output berbentuk garis lurus.

Pohon keputusan

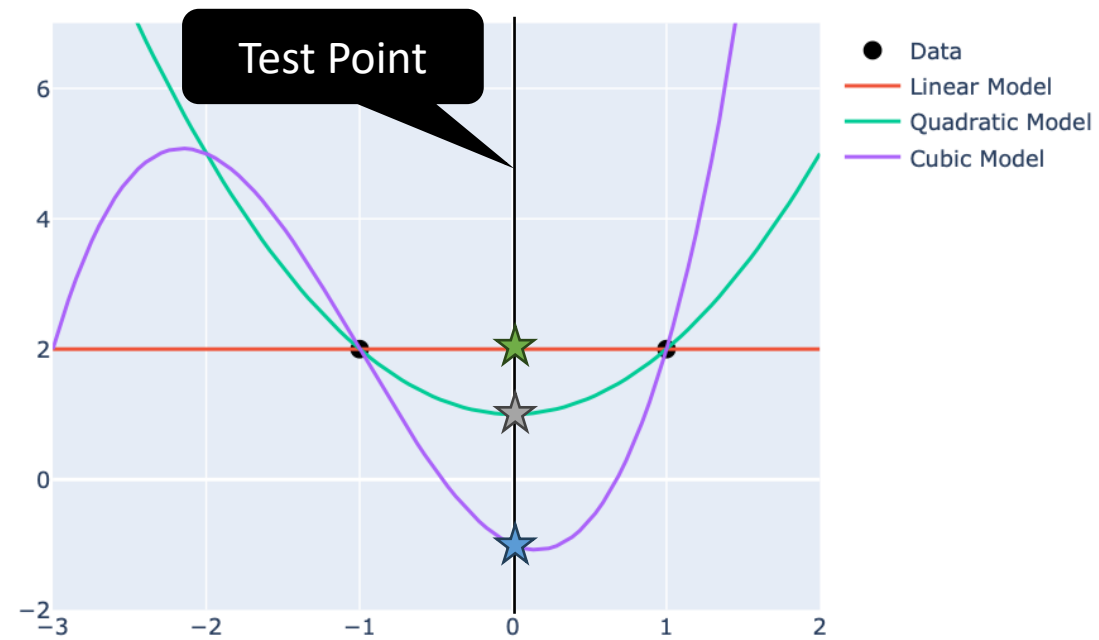
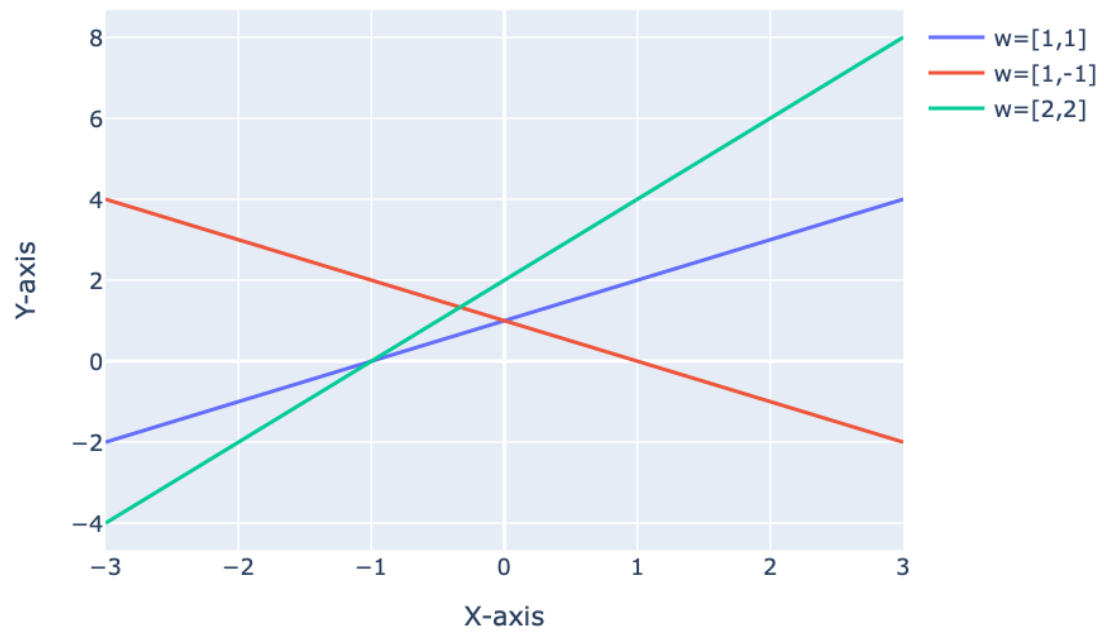
Mengasumsikan data dapat dipisahkan dengan aturan if–else sederhana.

Jaringan konvolusi

Mengasumsikan pola pada gambar bersifat lokal dan berulang di berbagai posisi.

Teorema “No Free Lunch”

Tidak ada satu model yang terbaik untuk semua persoalan. Tugas kita adalah memilih model yang bias induktifnya cocok dengan masalah yang dihadapi.



Model Linear vs Non-Linear

Semakin fleksibel belum tentu semakin baik.

Model Linear

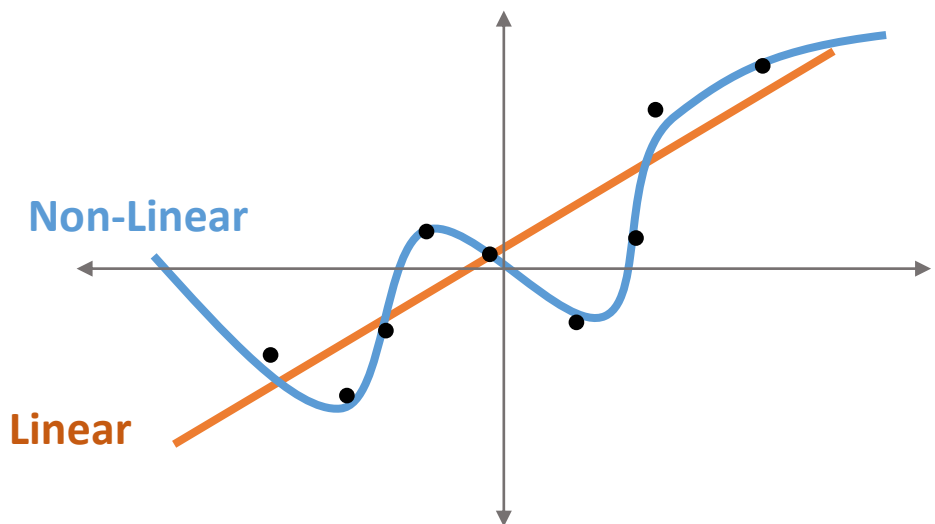
- Hanya dapat menangkap hubungan berbentuk garis atau bidang datar.
- Cepat dilatih, hemat data, dan mudah dijelaskan.
- Batas keputusan berupa garis lurus.

Model Non-Linear

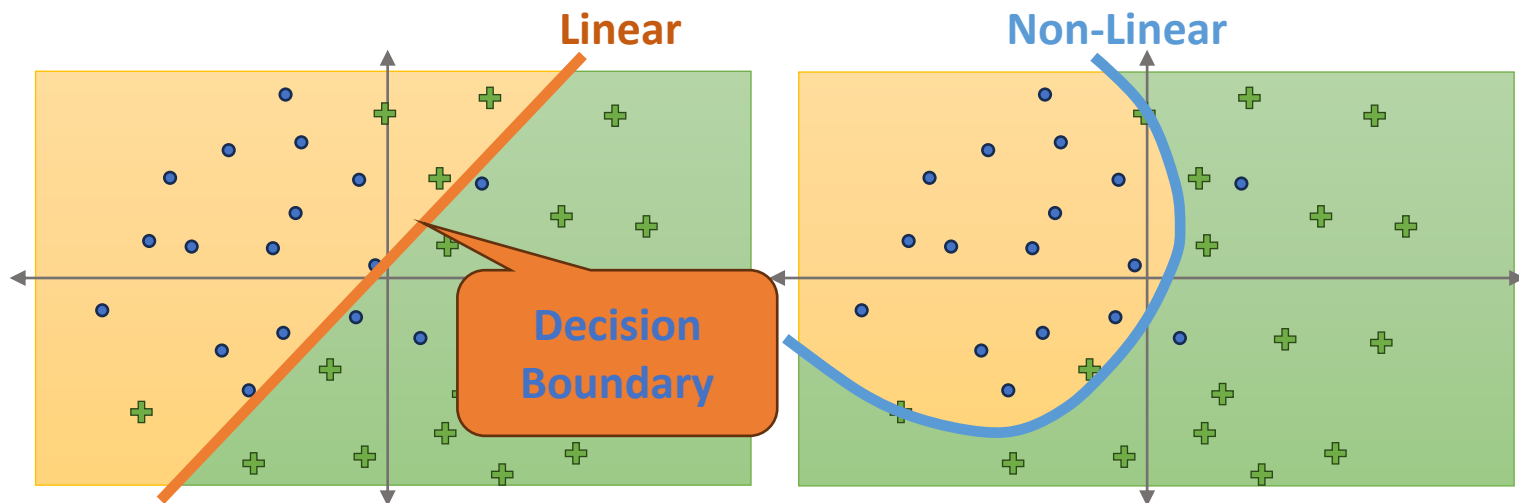
- Dapat membentuk kurva, percabangan, dan interaksi rumit antarfitur.
- Butuh lebih banyak data dan waktu komputasi.
- Lebih rawan overfitting dan lebih sulit ditafsirkan.

Analoginya: bila titik-titik data hampir membentuk garis lurus, penggaris sudah cukup. Bila polanya berkelok, barulah kita perlu alat yang lebih lentur — dengan konsekuensi tambahan.

Regression



Classification (Decision Boundary)



Kompleksitas: Underfitting dan Overfitting

Mencari titik seimbang antara terlalu sederhana dan terlalu rumit.

Model terlalu sederhana

Underfitting

- Gagal menangkap pola penting dalam data.
- Buruk di data latih MAUPUN data uji.
- Solusi: tambah kompleksitas atau perbaiki fitur.

Kompleksitas pas

Titik Seimbang

- Menangkap pola sebenarnya tanpa menghafal derau.
- Selisih akurasi latih dan validasi kecil.
- Inilah yang kita cari.

Model terlalu rumit

Overfitting

- Menghafal derau dan detail yang tidak penting.
- Sangat baik di data latih, buruk di data baru.
- Solusi: sederhanakan model, tambah data, regularisasi.

Analogi: menghafal soal latihan membuat nilai try out tinggi, tetapi begitu soal diubah sedikit, hasilnya berantakan.

Regularisasi

Menambahkan “hukuman” pada kerumitan model agar tidak terlalu mengikuti data latih.

Gagasan dasarnya

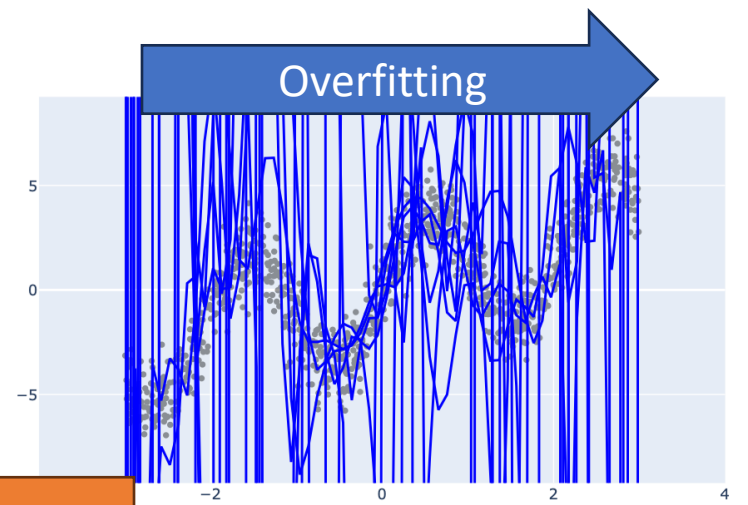
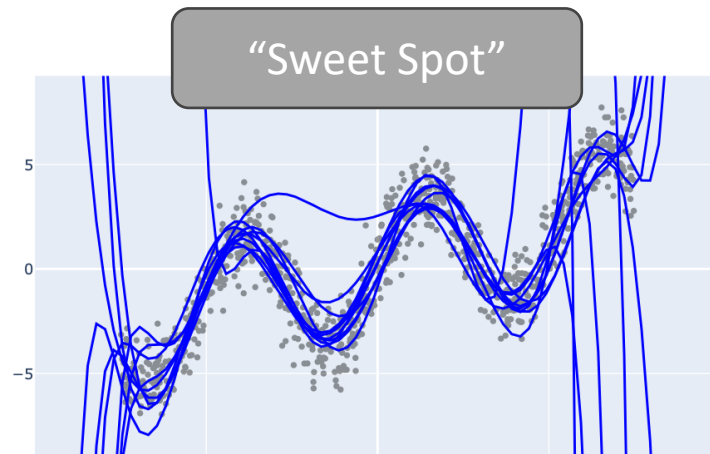
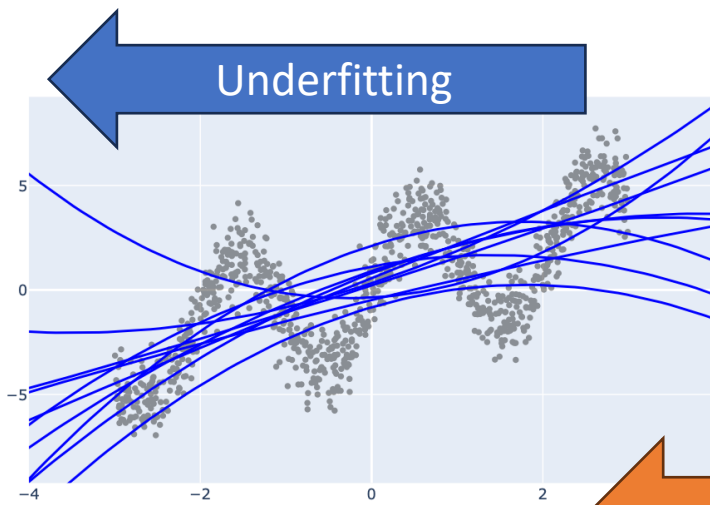
- Saat melatih, model tidak hanya diminta mengurangi kesalahan, tetapi juga menjaga agar bobotnya tetap kecil dan sederhana.
- Akibatnya model cenderung memilih pola yang halus dan bertahan pada data baru.

Semakin kuat regularisasi

- Terlalu lemah → model kembali overfitting.
- Terlalu kuat → model menjadi underfitting.
- Kekuatannya diatur lewat hyperparameter.

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression(C=0.1)    # C kecil = regularisasi lebih kuat
```

Menyetel kekuatan regularisasi termasuk tahap Optimisasi, yang dibahas pada materi berikutnya.



More Regularization

Model Parametrik vs Non-Parametrik

Perbedaannya terletak pada apakah jumlah parameter ikut bertambah seiring data.

Aspek	Parametrik	Non-Parametrik
Jumlah parameter	Tetap, ditentukan di awal	Bertambah seiring banyaknya data
Contoh	Regresi linear, regresi logistik	k-Nearest Neighbors, kernel SVM
Kecepatan pelatihan	Cepat	Lebih lambat, sering menyimpan seluruh data
Fleksibilitas	Terbatas pada bentuk yang diasumsikan	Tinggi, dapat mengikuti pola rumit
Kebutuhan data	Relatif sedikit	Banyak
Keterbacaan	Mudah ditafsirkan	Sering sulit ditafsirkan

k-NN adalah contoh khas non-parametrik: ia tidak merangkum data menjadi rumus, melainkan menyimpan seluruh contoh dan membandingkan data baru dengan tetangga terdekatnya.

Cara Memilih Keluarga Model

Urutan kerja yang aman dan hampir selalu berhasil.

1

Tentukan jenis masalahnya

Klasifikasi, regresi, klustering, atau reduksi dimensi. Ini langsung mempersempit pilihan.

2

Mulai dari model linear dengan fitur yang baik

Model linear ditambah rekayasa fitur adalah cara umum memasukkan pengetahuan awal, dan menjadi tolok ukur (baseline) yang jujur.

3

Naikkan kompleksitas secara bertahap

Coba pohon keputusan, ensemble, lalu jaringan saraf bila memang diperlukan.

4

Bandingkan dengan data validasi

Bukan dengan data latih dan bukan pula dengan data uji.

Materi ini berfokus pada model klasik di Scikit-Learn. Perancangan model deep learning dibahas pada pertemuan-pertemuan berikutnya.

Ringkasan Materi Ini

Enam gagasan kunci dari dua bagian yang sudah dibahas.

1

Rumuskan target, tujuan, dan data sebelum memilih algoritma apa pun.

2

Jenis data menentukan jenis pembelajaran: berlabel, tanpa label, atau berbasis imbalan.

3

Tujuan ML adalah generalisasi, bukan kecocokan dengan data latih.

4

Data dibagi menjadi latih, validasi, dan uji; data uji dipakai sekali di akhir.

5

Rekayasa fitur mengubah data mentah menjadi vektor angka yang bermakna.

6

Keluarga model menentukan ruang hipotesis dan bias induktif — inti dari kemampuan generalisasi.

SAMPAI JUMPA DI MATERI BERIKUTNYA

Terminologi dan Teknik Dasar (2)

Optimisasi · Prediksi dan Evaluasi

Materi diadaptasi dari kuliah “Machine Learning Mechanics — Terminology and Techniques” oleh Joseph E. Gonzalez dan Narges Norouzi. Rujukan buku: Bab 1.